

Reviewer Pack — Minimal Rank of Quasi-Metric Spaces Bounds Monotone Circuit ...

Entry ad66c1ea5d7c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 01:38:56 UTC

Minimal Rank of Quasi-Metric Spaces Bounds Monotone Circuit Size for k-CLIQUE

Entry ID: ad66c1ea5d7c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 01:38:56 UTC

1. Conjecture

Field A (mathematical branch): Quasi-metric Space Theory **Field B** (complexity object): Monotone Circuit Complexity

Statement:

{'text': 'The minimal rank of a quasi-metric space associated with an n -variable k-CLIQUE instance is $\Theta(n^k)$.', 'quantitative_relation': 'E[X(instance)] = $\Theta(n^k)$ '}

Rationale (proposer's reasoning):

{'text': 'Quasi-metric spaces provide a flexible framework to model the distance between elements in a set. By associating a quasi-metric space with a k-CLIQUE instance, we may capture the inherent complexity of the problem, leading to insights into monotone circuit lower bounds.'}

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f81945d4f9760d08

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the minimal ranks from 30 independent trials matches or exceeds n^k with a standard deviation $\leq 1/2 \cdot n^k$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-metric space minimal rank" AND "monotone circuit complexity"
- "k-CLIQUE instance quasi-metric space" AND circuit size" - " $\theta(n^k)$ bound quasi-metric space" AND monotone circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = Fraction(0, 1)
    for j in range(n):
```

```

        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def quasi_metric_space(edges):
    n = len(edges) + 1
    A = [[float('inf')] * n for _ in range(n)]
    for u, v in edges:
        A[u][v] = A[v][u] = random.random()
    for i in range(n):
        A[i][i] = 0

    # Ensure the matrix is symmetric and non-negative
    for i in range(n):
        for j in range(i+1, n):
            if A[j][i] < A[i][j]:
                A[i][j], A[j][i] = A[j][i], A[i][j]

    return A

def min_rank(A):
    gaussian_elimination(A)
    rank = 0
    for row in A:
        if any(x != 0 for x in row):
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    k = random.randint(2, min(n-1, 3))
    edges = list(combinations(range(n), 2))
    random.shuffle(edges)
    edges = edges[:k*(n-k)]

    A = quasi_metric_space(edges)
    rank = min_rank(A)

    # Calculate monotone circuit size (simplified heuristic)
    circuit_size = n * k

    return {
        "metric_name": "min_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= n**k - Fraction(1, 2) * n**k and rank <= n**k + Fraction(1, 2) * n**k,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1000, 9999) for _ in range(30)]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_rank = sum(r["metric_value"] for r in results) / len(results)
std_rank = (sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='rank does not match n^k' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fbe3ebd.py", line 105, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fbe3ebd.py", line 86, in run_trial
    rank = min_rank(A)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fbe3ebd.py", line 69, in min_rank
    gaussian_elimination(A)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fbe3ebd.py", line 29, in gaussian_elimination
    factor = Fraction(A[j][i], A[i][i])
                ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete the required 30 independent trials to evaluate the conjecture. | next: Re-run the test ensuring that it completes all 30 trials and produces the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11432
2	propose	ollama_remote	glm4:latest	0	0	10097
3	preregistration	ollama_remote	glm4:latest	0	0	5810
4	novelty	ollama_remote	glm4:latest	0	0	4704
5	novelty	ollama_remote	glm4:latest	0	0	5508
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40874
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13386
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13389
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11726
10	judge	ollama_remote	glm4:latest	0	0	16939

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133866 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ad66c1ea5d7c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ad66c1ea5d7c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ad66c1ea5d7c.tar.gz` (if generated)